National University of Computer and Emerging Sciences Islamabad
Operating System Lab Spring 2025

Lab Task: Process Creation using Fork and Exec System Calls

Fork Visualizer for Practice

Before diving into the tasks, it's beneficial to familiarize yourself with the `fork()` system call and how processes are created. You can practice and visualize the process creation flow using the following online Fork Visualizer:

[Online Fork Visualizer]

This tool allows you to simulate the behavior of the `fork()` system call in a graphical way, helping you better understand how parent and child processes are created and interact.

Lab Tasks

Task 1: Using `execl()` to Execute a New Program

Objective:
Write a program that:

- Creates a child process using `fork()`.
- The child process uses `execl()` to replace itself with a new program (`/bin/ls`) to list the contents of the current directory.
- The parent process waits for the child process to complete before printing "Child process completed".

Expected Output (Example):

```
Files in directory:
file1.txt  file2.cpp  main.c
Child process completed.
```

Task 2: Using `execvp()` to Run a User-Specified Command

Objective:
Write a program where:

- The child process asks the user for a shell command (e.g., `ls`, `pwd`, `date`).
- It then uses `execvp()` to execute the command.
- The parent process waits for the child to finish and then prints "Command execution finished".

Example Input/Output:

```
Enter a command: date
Mon Feb 12 10:15:45 UTC 2025
Command execution finished.
```

Task 3: Using `execv()` to Run a Custom Program

Objective:

- Create a separate C program (`child_program.c`) that prints "Hello from child process!".
- In the main program, create a child process using `fork()`.
- The child process uses `execv()` to run `child_program`.
- The parent process waits for the child to complete before printing "Execution completed".

Expected Output:

```
Hello from child process!
Execution completed.
```

Task 4: Using `execle()` with Custom Environment Variables

Objective:

- The child process sets a custom environment variable (`MY_VAR="Hello from execle"`) and executes a program (`env_printer`) using `execle()`.
- The `env_printer` program should print the value of `MY_VAR`.

- The parent process waits for the child to finish before exiting.

Example Output:

```
MY_VAR: Hello from execle
Child process completed.
```

Task 5: Using `fork()` - Four Child Processes Performing Different Tasks

Objective:
The parent process creates **four child processes**, each performing a different task.

Steps:

1. **Parent process** creates four child processes using `fork()`.
2. Each child performs a different task:
    - **Child 1:** Prints the current system date and time (`date`).
    - **Child 2:** Lists the current directory (`ls`).
    - **Child 3:** Calculates the factorial of a user-provided number.
    - **Child 4:** Reads a number from the user and prints whether it's even or odd.
3. The **parent process** waits for all children to complete before printing `"All child processes completed"`.

Expected Input/Output (Example):

```
Child 1: Current date and time: Mon Feb 12 10:30:00 UTC 2025
Child 2: Directory contents:
file1.txt  file2.cpp  main.c
Child 3: Enter a number: 5
Child 3 result (Factorial): 120
Child 4: Enter a number: 8
Child 4 result: Even
All child processes completed.
```

Deliverables:

- **Submit the c code for each of the tasks separately (in .cpp files).**
- **Provide screenshots or output showing the results of running the programs**

THE END
